

INFORME Y REFACTORIZACIÓN - IVÁN (1º DAM)

Buenas. En este documento dejo los fallos que he encontrado en el código del proyecto **TaskMaster** mientras hacía las pruebas del Reto 6, cómo los he arreglado para que no pete la aplicación y las pruebas que he hecho con JUnit.

1. FALLOS DETECTADOS

Fallo 1: El if (conn == null)

- **Dónde estaba:** En TareaDAO.java, dentro de los métodos de crearTarea y cambiarEstadoTarea.
- **El problema:** En el código que tenía, primero ejecutaba la consulta a la base de datos (stmt.executeUpdate()) y *después* ponía la línea de if (conn == null) return. Esto no tiene sentido porque si la conexión de verdad venía vacía (null), el programa iba a explotar líneas antes al intentar preparar la sentencia (PreparedStatement), por lo que el programa se cerraba con error y nunca llegaba a leer ese if.
- **Cómo lo he arreglado:** He quitado ese if del final porque estaba mal puesto. Como ahora usamos el try-with-resources, si la conexión falla o devuelve un nulo, salta directamente al bloque catch (SQLException e) del principio y muestra el mensaje de error por la consola sin que se cierre la aplicación de golpe.

Fallo 2: El Array de Categorías era fijo

- **Dónde estaba:** En TareaDAO.java, en el método obtenerCategorias().
- **El problema:** Estaba puesto un array con un tamaño fijo de 10 (Categoria[] categorias = new Categoria[10]).
 1. Si en la base de datos el profesor o yo metemos 11 categorías, la aplicación da un error de sobre tamaño y se rompe.
 2. Si solo tenemos 2 o 3 categorías, las otras 7 posiciones gastan memoria.
- **Cómo lo he arreglado:** He cambiado el array de toda la vida por un ArrayList dinámico (List<Categoria> categorias = new ArrayList<>()). Así da igual las categorías que haya en la base de datos.

2. OPTIMIZACIÓN DE LA BBDD

- **Lo que había antes:** En el método mostrarTareas, la consulta tenía un `SELECT *`:

```
SELECT * FROM tareas inner join estados on estados.id_estado = tareas.id_estado  
where id_usuario = ?;
```

- **El problema:** El asterisco (*) lo pide todo, incluyendo descripciones muy largas y observaciones que no hacen falta para el menú de la consola. Esto hace que la base de datos vaya más lenta porque tiene que mover muchos datos innecesarios entre MySQL y Java.
- **Cómo lo he arreglado:** He cambiado el asterisco por los campos exactos que se van a pintar en los SOUT (el id, el título y el nombre del estado). La consulta optimizada se queda así:

```
SELECT t.id_tarea, t.titulo, e.nombre  
  
FROM tareas t  
  
INNER JOIN estados e ON e.id_estado = t.id_estado  
  
WHERE t.id_usuario = ?;
```

3. PRUEBAS UNITARIAS CON JUNIT

He creado un par de casos de prueba en la carpeta de test para comprobar que la lógica de las tareas no falle en cosas raras:

Prueba 1: Controlar las fechas al revés

- **Qué prueba:** Que no te deje crear una tarea si pones que la fecha de vencimiento es anterior a la fecha en la que la has creado (por ejemplo, crearla hoy pero que venza ayer).
- **Cómo lo hace:** He metido un método en Tarea.java llamado `esFechaValida()`. El test le pasa una tarea con la fecha de creación en 2026-06-14 y vencimiento en 2026-06-10. El test pasa correctamente porque el método devuelve false (detecta el error).

Pruebas de cambiar estado a algo que no existe

- He modificado el método de cambiar estado para que si intentas cambiar el estado de una tarea introduciendo un ID que no está en la base de datos, el UPDATE de SQL devuelva 0 filas afectadas y pinte un mensaje de *"No se ha encontrado la tarea"* en vez de quedarse colgado.

4. CONCLUSIÓN

Con estos cambios el código ya no tiene los fallos de los arrays fijos que se podían romper, las consultas SQL van más ligeras porque no piden datos de más y la consola es más segura gracias a los try-catch bien ordenados. Todo está subido al repositorio de GitHub.